FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

4

IMPLEMENTING KNOWLEDGE REPRESENTATION, RULE-BASED REASONING (RBR), AND CASE-BASED REASONING (CBR)

Student Name	ONG-IKO, KATELYN M.
Section:	AN43
Professor:	MS. CRISOLA TAN
Date:	11/2/2026

I. BACKGROUND INFORMATION

Knowledge Representation (KR) is the method of organizing facts, rules, and relationships so an AI system can understand and reason about the world.

Reasoning (R) allows the system to infer, make decisions, and solve problems based on that knowledge. Two major reasoning approaches from the module:

1. Rule-Based Reasoning (RBR)

Uses IF–THEN rules, a Knowledge Base, Working Memory, and an Inference Engine.

2. Case-Based Reasoning (CBR)

Solves new problems using past cases using the 4R Cycle:

Retrieve → Reuse → Revise → Retain

This laboratory lets you build small working implementations of both reasoning methods.

II. TECHNICAL ASSESSMENT

IMPLEMENTING KNOWLEDGE REPRESENTATION, RULE-BASED REASONING (RBR), AND CASE-BASED REASONING (CBR)

Objective

At the end of this laboratory activity, students are expected to:

1. Construct a simple knowledge base using Python data structures.
2. Implement a basic Rule-Based Reasoning (RBR) system using IF–THEN logic.
3. Build a simple Case-Based Reasoning (CBR) system using similarity matching.
4. Compare how RBR and CBR behave when solving the same type of problem.
5. Document observations and insights on KR, RBR, and CBR.

Materials Needed

1. Hardware
   - A laptop or desktop computer
   - Stable internet connection (for research, documentation, submission)
2. Software

A programming environment:

- Python (recommended: IDLE, VS Code, Jupyter Notebook, PyCharm)
- Any text editor (Notepad++, Sublime, VS Code) if needed

3. File Formats & Tools

Word processor for documentation:

- MS Word, Google Docs, or LibreOffice
- Screenshot tool (e.g., Snipping Tool, Lightshot, Print Screen)

General Instructions

1. This **Technical Assessment (TA3)** must be completed by Pair.
2. Include clear comments in your code to describe its logic.
3. Each program should display the correct input, process, and output.
4. Ensure that your script displays clear and readable outputs.
5. Screenshot of program output for RBR
6. Screenshot of program output for CBR
7. Reflection answers
8. Short description of your knowledge base and use case
9. Compile all Python codes and corresponding outputs into one PDF file.
10. Save it as **TA4_lastname1.pdf**
    (Example: TA2_Patilano.pdf)
11. Submit the **python file** and **PDF** file.

LABORATORY TASKS

PART 1 — Knowledge Representation (KR) Setup

Task 1A — Create a Knowledge Base

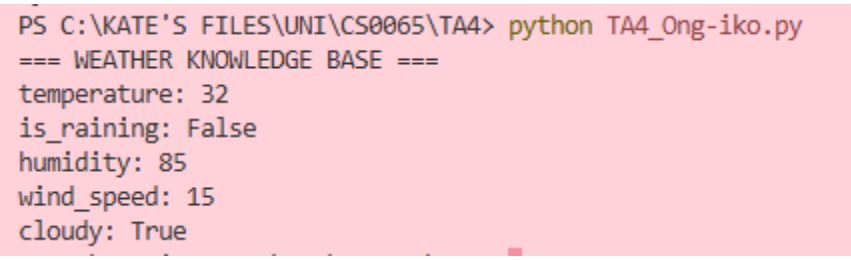
Create a Python dictionary storing at least:

- 5 facts about an object (e.g., weather, appliances, students, medical symptoms)
- 3 categories (e.g., temperature, status, presence, actions)

Example structure to follow:

```
facts = {
    "temperature": 32,
    "is_raining": False,
    "humidity": 85,
}
```

OUTPUT:



```
PS C:\KATE'S FILES\UNI\CS0065\TA4> python TA4_Ong-iko.py
=== WEATHER KNOWLEDGE BASE ===
temperature: 32
is_raining: False
humidity: 85
wind_speed: 15
cloudy: True
```

PART 2 — Implement Rule-Based Reasoning (RBR)

Task 2A — Create at least 3 IF–THEN rules

Write Python functions representing rules.

Example:

```
def rule_hot(facts):
    if facts["temperature"] > 30:
        return "Turn on the aircon"
```

Task 2B — Implement an Inference Engine

Create a loop that:

1. Reads the knowledge base
2. Checks each rule
3. Returns all actions inferred

Task 2C — Display Output

Example [Program should print:]

Rule-Based Reasoning Actions:
- Turn on the aircon
- Bring umbrella

```
PS C:\KATE'S FILES\UNI\CS0065\TA4> python TA4_Ong-iko.py
=== WEATHER KNOWLEDGE BASE ===
temperature: 32
is_raining: False
humidity: 85
wind_speed: 15
cloudy: True

=== RULE-BASED REASONING ACTIONS ===
- Turn on the air conditioner.
- Use a dehumidifier.
- Expect possible rain.
```

PART 3 — Implement Case-Based Reasoning (CBR)

Task 3A — Build a Case Base

Create at least **3 past cases**, each containing:

- A set of problem features
- A matching solution

Example:

```
case_base = [
    {"problem": {"won't turn on": True, "noise": False},
     "solution": "Check power cable."}
]
```

Task 3C — Reuse Step

Return the solution of the most similar case.

OUTPUT (TASKS 3A-3C):

```
PS C:\KATE'S FILES\UNI\CS0065\TA4> python TA4_Ong-iko.py
=== WEATHER KNOWLEDGE BASE ===
temperature: 32
is_raining: False
humidity: 85
wind_speed: 15
cloudy: True

=== RULE-BASED REASONING ACTIONS ===
- Turn on the air conditioner.
- Use a dehumidifier.
- Expect possible rain.

=== CASE-BASED REASONING ===
Most Similar Past Case:
{'temperature': 35, 'is_raining': False, 'humidity': 90}

Reused Solution:
Turn on the air conditioner and use a dehumidifier.
```

Task 3D — Revise Step

Allow student to adjust solution manually or return unchanged.

OUTPUT:

```
PS C:\KATE'S FILES\UNI\CS0065\TA4> python TA4_Ong-iko.py
=== WEATHER KNOWLEDGE BASE ===
temperature: 32
is_raining: False
humidity: 85
wind_speed: 15
cloudy: True

=== RULE-BASED REASONING ACTIONS ===
- Turn on the air conditioner.
- Use a dehumidifier.
- Expect possible rain.

=== CASE-BASED REASONING ===
Most Similar Past Case:
{'temperature': 35, 'is_raining': False, 'humidity': 90}

Reused Solution:
Turn on the air conditioner and use a dehumidifier.

=== REVISE SOLUTION ===
Do you want to modify the solution? (yes/no): yes
Enter your revised solution: Turn on the air conditioner, use a dehumidifier, and drink plenty of water to stay hydrated.
Final Solution: Turn on the air conditioner, use a dehumidifier, and drink plenty of water to stay hydrated.
```

Task 3E — Retain Step

Add the new case to the case base.

OUTPUT:

```
PS C:\KATE'S FILES\UNI\CS0065\TA4> python TA4_Ong-iko.py
=== WEATHER KNOWLEDGE BASE ===
temperature: 32
is_raining: False
humidity: 85
wind_speed: 15
cloudy: True

=== RULE-BASED REASONING ACTIONS ===
- Turn on the air conditioner.
- Use a dehumidifier.
- Expect possible rain.

=== CASE-BASED REASONING ===
Most Similar Past Case:
{'temperature': 35, 'is_raining': False, 'humidity': 90}

Reused Solution:
Turn on the air conditioner and use a dehumidifier.

=== REVISE SOLUTION ===
Do you want to modify the solution? (yes/no): no
Final Solution: Turn on the air conditioner and use a dehumidifier.

=== RETAINED NEW CASE ===
New case successfully added to the case base!
Total number of cases: 4
PS C:\KATE'S FILES\UNI\CS0065\TA4>
```

PART 4 — Comparative Reflection

Write a short paragraph answering:

1. Which system (RBR or CBR) was easier to implement? Why?
   Rule-Based Reasoning (RBR) was easier to implement because it only required simple IF–THEN conditions. It was straightforward to create rules and check whether the conditions were met.

2. Which is more flexible in solving unstructured problems?
   Case-Based Reasoning (CBR) is more flexible because it can use previous experiences to solve new problems, even when they are not exactly the same.

3. Which is better for situations with continuously changing data?
   Rule-Based Reasoning (RBR) is useful when decisions need to follow updated facts and predefined conditions. However, CBR can also adapt by storing new cases and their solutions.

4. Examples of real-world applications for both.
   - RBR: Medical diagnosis systems, automated customer support, and smart home automation.
   - CBR: Customer service troubleshooting, medical case comparisons, and technical equipment repair.

Your Code - Laboratory Tasks

```
# PART 1: KNOWLEDGE REPRESENTATION (KR)

# Knowledge Base: Stores current weather information
facts = {
    "temperature": 32,
    "is_raining": False,
    "humidity": 85,
    "wind_speed": 15,
    "cloudy": True
}

# Display the Knowledge Base
print("=== WEATHER KNOWLEDGE BASE ===")
for category, value in facts.items():
    print(f"{category}: {value}")

# PART 2: RULE-BASED REASONING (RBR)

# Rule 1: Check if the temperature is hot
def rule_hot(facts):
    if facts["temperature"] > 30:
        return "Turn on the air conditioner."

# Rule 2: Check if it is raining
def rule_rain(facts):
    if facts["is_raining"]:
        return "Bring an umbrella."

# Rule 3: Check if humidity is high
def rule_humidity(facts):
    if facts["humidity"] > 80:
        return "Use a dehumidifier."

# Rule 4: Check if it is cloudy
def rule_cloudy(facts):
    if facts["cloudy"]:
```

```
        return "Expect possible rain."

# List of available rules
rules = [
    rule_hot,
    rule_rain,
    rule_humidity,
    rule_cloudy
]

# Task 2B: Inference Engine
def inference_engine(facts, rules):
    actions = []

    # Check every rule against the knowledge base
    for rule in rules:
        result = rule(facts)

        if result:
            actions.append(result)

    return actions

# Task 2C: Display Output
print("\n=== RULE-BASED REASONING ACTIONS ===")

actions = inference_engine(facts, rules)

for action in actions:
    print("-", action)

# PART 3: CASE-BASED REASONING (CBR)

# Task 3A: Store previous weather cases
case_base = [
    {
        "problem": {
            "temperature": 35,
            "is_raining": False,
            "humidity": 90
        },
        "solution": "Turn on the air conditioner and use a dehumidifier."
    },
    {
```

```
        "problem": {
            "temperature": 25,
            "is_raining": True,
            "humidity": 75
        },
        "solution": "Bring an umbrella and stay indoors."
    },
    {
        "problem": {
            "temperature": 20,
            "is_raining": False,
            "humidity": 60
        },
        "solution": "Enjoy the cool weather outdoors."
    }
]

# Task 3B: Retrieve the most similar case
def retrieve_case(new_problem, case_base):
    best_case = None
    highest_similarity = -1

    for case in case_base:
        similarity = 0

        # Compare the weather features
        for feature in new_problem:
            if new_problem[feature] == case["problem"][feature]:
                similarity += 1

        if similarity > highest_similarity:
            highest_similarity = similarity
            best_case = case

    return best_case

# Task 3C: Reuse the solution
print("\n=== CASE-BASED REASONING ===")

# Use the current weather facts as the new problem
new_problem = {
    "temperature": facts["temperature"],
    "is_raining": facts["is_raining"],
    "humidity": facts["humidity"]
```

```
}

retrieved_case = retrieve_case(new_problem, case_base)

print("Most Similar Past Case:")
print(retrieved_case["problem"])

solution = retrieved_case["solution"]

print("\nReused Solution:")
print(solution)

# Task 3D: Revise the solution
print("\n=== REVISE SOLUTION ===")

choice = input("Do you want to modify the solution? (yes/no): ")

if choice.lower() == "yes":
    solution = input("Enter your revised solution: ")

print("Final Solution:", solution)

# Task 3E: Retain the new case
new_case = {
    "problem": new_problem,
    "solution": solution
}

case_base.append(new_case)

print("\n=== RETAINED NEW CASE ===")
print("New case successfully added to the case base!")
print("Total number of cases:", len(case_base))
```